

# Path planning using Hybrid-State A\* for driverless vehicles

Matteo De Rosa

May 11, 2018

# Contents

|          |                                  |           |
|----------|----------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>              | <b>3</b>  |
| <b>2</b> | <b>Background</b>                | <b>4</b>  |
| 2.1      | Cardiff Racing . . . . .         | 4         |
| 2.2      | Existing Solutions . . . . .     | 5         |
| 2.3      | Hybrid-State A* search . . . . . | 6         |
| 2.3.1    | Application . . . . .            | 6         |
| 2.3.2    | Algorithm outline . . . . .      | 6         |
| 2.3.3    | Expected behaviour . . . . .     | 7         |
| 2.4      | Dubins path . . . . .            | 7         |
| 2.5      | Tools and Technologies . . . . . | 8         |
| <b>3</b> | <b>Problem Statement</b>         | <b>9</b>  |
| 3.1      | Execution Time . . . . .         | 9         |
| 3.2      | Goal specification . . . . .     | 10        |
| 3.2.1    | Mono Goal . . . . .              | 10        |
| 3.2.2    | Waypoints . . . . .              | 11        |
| <b>4</b> | <b>Implementation</b>            | <b>12</b> |
| 4.1      | Hybrid-State A* . . . . .        | 12        |
| 4.1.1    | Motion Primitives . . . . .      | 12        |
| 4.1.2    | Open and Closed lists . . . . .  | 13        |
| 4.1.3    | Heuristic and Cost . . . . .     | 14        |
| 4.1.4    | Defining the Goal . . . . .      | 16        |
| 4.1.5    | Collision Detection . . . . .    | 16        |
| 4.1.6    | Dubins path . . . . .            | 16        |
| 4.1.7    | Minor notes . . . . .            | 18        |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>5</b> | <b>Results and Evaluation</b>                        | <b>19</b> |
| 5.1      | Specification . . . . .                              | 19        |
| 5.2      | Finding optimal distance between waypoints . . . . . | 19        |
| 5.2.1    | Straight track . . . . .                             | 20        |
| 5.3      | Testing performance . . . . .                        | 21        |
| 5.3.1    | Circular track without Dubins path ext . . . . .     | 21        |
| 5.4      | Fail case . . . . .                                  | 24        |
| <b>6</b> | <b>Future Work</b>                                   | <b>26</b> |
| 6.0.1    | First focus . . . . .                                | 26        |
| 6.0.2    | Second focus . . . . .                               | 26        |
| 6.0.3    | Other potential improvements . . . . .               | 26        |
| <b>7</b> | <b>Conclusions</b>                                   | <b>28</b> |
| <b>8</b> | <b>Reflection</b>                                    | <b>29</b> |

# Chapter 1

## Introduction

Currently a lot of funds are being placed in the field of autonomous driving and the field of robotics. The projects aims at implementing a solution to the path planning problem to fit in part of a larger project to have a driverless racing car compete 2.1. Along this project, I will spend time researching current solutions applied within the field of robotics, autonomous driving and racing.

My goal for this project is to have fully implemented an algorithm that can generate a path from a given track, having a skeleton or simulated vehicle capable of successfully following the path, by feeding data to and from the simulation to the algorithm. My other goal for this project is to research the capabilities, benefits and disadvantages from my chosen implementation of the path planning algorithm, making a minor scientific contribution.

# Chapter 2

## Background

### 2.1 Cardiff Racing

This project aims at developing a solution to a subset of problems within a larger project taken on by Cardiff Racing AI, a team of academics with aim of developing a driverless racing software system with intent to compete in the formula student AI controlled competition organised by IMechE. The system currently designed by the team is split into five components:

1. Cone Detection
2. Simultaneous Localization and Mapping (SLAM)
3. Path planning
4. Action formulation
5. Simulated Environment

The project aims tackle the path planning component of this system. The path planning component will be required to interface with the SLAM component and the Action Formulator. SLAM will feed a model of the world to the path planner, which constitutes track boundaries and the coordinates of the vehicle. From here the Path planning component aims to construct a path which is fed the action formulator can send controls for the car to follow along.

To calculate and derive racing lines for the car to follow we will first need a full or a large portion of the track modelled, due to the format of the competition we cannot feed a complete model of the race track before we begin the race and no first lap is first made to gather that initial model. This leads to having the car having to plan its path within an unknown and incomplete environment.

## 2.2 Existing Solutions

The path planning problem can often be solved by discretizing the world into nodes or states in which you can then use search algorithms like that of A\* and D\* to find the shortest path to a goal, the problem with these algorithms is that they do not consider the non-holonomic nature of the car.

A more fitting solution is the use of rapidly-exploring random trees (RTT) although this algorithm can be adapted to produce viable non-holonomic paths [1], since the expansion of the trees is random it causes the expansion nodes to be in all directions from the root node See figure 2.1, this leads to a lot of unnecessary exploring and computation to find a close to optimal solution, since the flow of a track is unidirectional it is extremely unlikely that we will ever have to reverse or have to change direction by a very large degree.

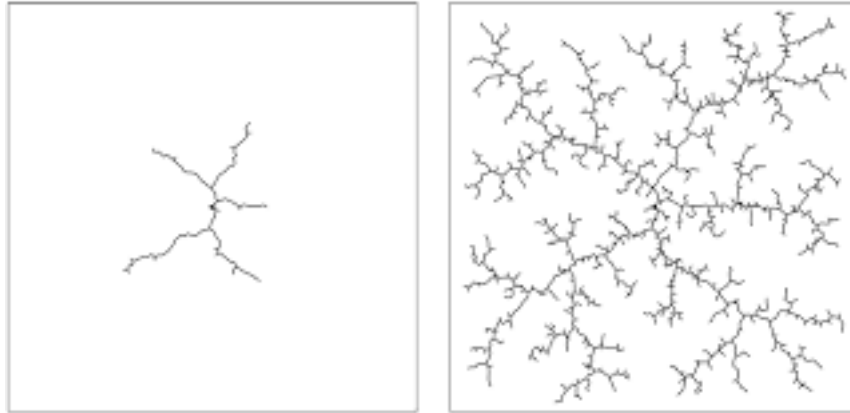


Figure 2.1: Graphical representation of the nature of Randomly-Exploring Random Trees

## 2.3 Hybrid-State A\* search

### 2.3.1 Application

One approach to the problem involves the use Hybrid-State A\* search which is an adaptation of the standard a\* search algorithm, where unlike a\*, it can deal with the continuous nature of the search space representing the real world. Hybrid A\* was used effectively for getting a car to navigate and perform complex manoeuvres in urban environments, such as reversing into parking spots [2]. Hybrid A\* was also used for successfully driving Stanford's vehicle "Junior" in the DARPA Urban Challenge [3].

My project will aim to implement this solution, for my requirements it is not necessary that the car can perform complex manoeuvres like that of reversing into tight parking spots, but nonetheless, it is fitting to the problem at hand as it can produce a trajectory that is within the neighbourhood of the global optimum.

### 2.3.2 Algorithm outline

Hybrid A\* requires the discretizing the map into  $(x, y)$  cells forming a grid, this grid is then used to close off search space when a cell is expanded by a node, just like standard a\*, but it is possible for multiple nodes lie within an expanded cell, for clarification See section 4.1.2.

An open list  $O$  is used to store nodes that have been expanded and might need further exploration, once a node is conclusively processed it is placed in the closed list  $C$ . Nodes can be interpreted with  $(x, y)$  coordinates and heading  $\theta$ , See figure 2.2, the starting position and orientation of the car will be the root node. From a node a set of new nodes can be calculated from a pre-defined travelling distance and a set of steering angles, also known as motion primitives, the number of new nodes depends on the number of frequency of discrete steering angles. The new nodes keep a pointer to their parent node  $p$  and are then added to an open list. The cost  $f(n) = g(n) + h(n)$  is computed for each node, where  $g(n)$  is the cost required to reach the node, in our case the distance travelled, and  $h(n)$  being the heuristic which is our estimate of the minimum cost from the goal. The node with the lowest  $f(n)$  is chosen from the open list and the process from calculating motion primitives is repeated until the shortest path to the goal is found.

To summarise a node  $n$  can be fully described by  $n = (x, y, \theta, g, f, p)$

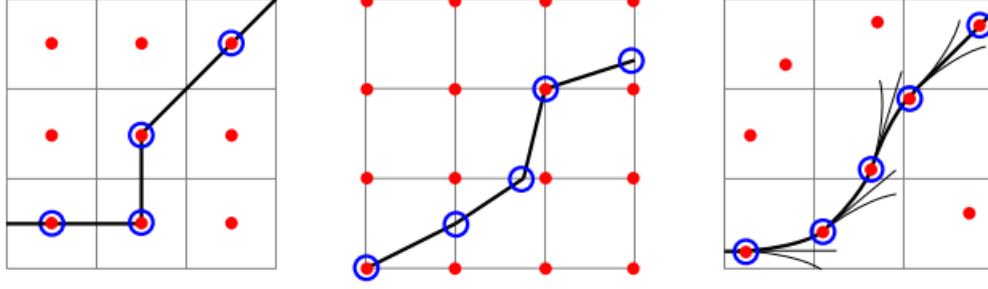


Figure 2.2: Comparison of Search Algorithms, Nodes are represented by the red dots, Left: A\*, Centre: D\*, Right: Hybrid-State A\*. [4]

### 2.3.3 Expected behaviour

Due to the discretization of the steering angle range, it is unlikely for Hybrid-State A\* to find the global optimum, but the solution is likely to be within the neighbourhood of it. This typically also leads to producing a path that is sub-optimal and not smooth, where unnecessary turns are made along the path. To improve the produced path, this project will aim to post-process the produced path by performing some local optimization and smoothing.

## 2.4 Dubins path

Dubins path is the shortest curve that connects two points in x-y space, with the constraints on the curvature of the path, the tangents of the initial and final points of the path and the vehicle that is travelling along the path cannot reverse. If the vehicle is able to reverse, then the path can follow the Reeds-Shepp curve. Dubins path does not take obstacles or barriers into consideration.



## 2.5 Tools and Technologies

I implemented the algorithms described using MATLAB R2017b, I used the Udacity simulator in Unity 2017.3 in attempt to test the effectiveness of the algorithm, using Python 3.5.2 to communicate between MATLAB and the Simulation.

# Chapter 3

## Problem Statement

### 3.1 Execution Time

During the initial lap of the track, the model of the world will start to grow, and so a new goal can be placed for the vehicle to aim for, this means that paths have to be constructed frequently, and the time required to do so needs to be minimal. One problem is that I do not have the exact requirement for the minimum execution time of my code, all I can do is make an ambitious estimate, and aim for that by minimizing execution time shaving off execution time where ever possible within my implementation.

One way to approach the problem is to measure execution time across with increasing distance from the goal, and with different configurations of goal placements, using these times I can find out how far the vehicle will travel when moving at different velocities, this will tell me how far and where the vehicle will stray away from the initial point of the generated path. Using these estimates potentially as a way of replotting the initial position of the path. There comes a certain point where execution time is too slow for high velocities because the vehicle will go off track before it can make change of course. Even though I can estimate how far the vehicle will travel away from the actual path, I cannot gauge the magnitude of the effect. For this reason, I've attempted to connect the simulation to MATLAB and have a controller try and follow the path provided.

## 3.2 Goal specification

### 3.2.1 Mono Goal

Goal placement is a critical component to the effectiveness of the algorithm, the distance of the goal is correlated to the execution time of the algorithm, another factor is the placement of the goal in relation when using Euclidian distance as a heuristic, See section 4.1.3.

There are two approaches to planting goals, the first approach involves placing the goal directly behind the starting point as this is the point where the vehicle would arrive after making a full lap of the track, but this leads to offloading the work to the heuristic. To have the heuristic encourage exploration away from the goal located just behind the starting point, the heuristic has to first take track boundaries into consideration, then it has to programmatically insert a simulated track boundary so its shortest distance is not from reversing, but rather to go around the visible track, See figure 3.1.

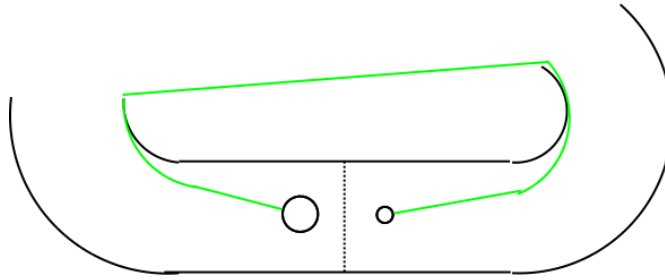


Figure 3.1: A graphical representation of an optimal heuristic for reaching the goal with incomplete track information. The green line is an estimate of an appropriate heuristic, the dotted line represents the simulated boundary, the small circle representing the starting position and the large circle representing the goal.

A few problems arise with this method, every time more of the track is discovered the current path needs to be checked for collisions. If a collision occurs then a new path needs to be generated, this can result in higher amount time recalculating the path, but if the execution time for generating the path is rather small this may not be a large problem. Another problem

arises when racing on a figure eight track, as now there are 3 potential routes for the generated path to fall into, and this will likely be the one leading straight back towards the goal, so one approach to solving this would be similar to what was suggested earlier by placing simulated barriers alongside the track using a method of taking the path with least resistance.

### **3.2.2 Waypoints**

The other approach involves placing goals along the discovered track that act as waypoints. Waypoints need to be placed at the centre of the track, since the heading or direction in which the vehicle must travel next cannot be determined, this also creates the most leeway.

There is a trade-off in the distance between waypoints, having a short distance between waypoints leads to having a path that is stretched towards the centre line, leading to no or minimal improvement over driving along the centre line. Having waypoints that are too far apart, when using Euclidian distance as the heuristic, can result in exponential increases in execution time, see subsection 5.3.1, and not solving the previously mentioned problem of the figure eight track.

This report will aim to find an optimal configuration for waypoint placement such as distance given my performance of my implemented algorithm, this is can be seen in the Results and Evaluation section of the report.

There is still another problem worth mentioning but is out of the scope of this project. The waypoint strategy relies on the vision range of the cone detection, in other words, how much of the track can be seen ahead of the car and built into the model of the track, if the range is not large enough then this approach risks falling into the problem mentioned earlier regarding the short distance between waypoints.

# Chapter 4

## Implementation

### 4.1 Hybrid-State A\*

#### 4.1.1 Motion Primitives

The path created is provably executable up to the degree of the motion model accuracy. And the motion model is set of calculations used to construct motion primitives, defined by arcs which follow certain restraints:

1. The distance travelled  $d$  has to be enough to exit the cell, for a cell of size  $s$  then  $d \geq \sqrt{2} \cdot s$
2. The curvature of a circle radius  $r$  is restricted by the maximum vehicle steering angle  $\alpha$

When constructing a motion primitive  $m$ , the turning angle  $\beta$  is calculated where  $l$  being the length of the vehicle:

$$\beta = \frac{d}{l * \tan(\alpha)}$$

If  $\beta > 0.0001$  we use the following equations

$$\begin{aligned} r &= d/\beta \\ \Delta x &= x - \sin(\theta) \cdot r \\ \Delta y &= y - \cos(\theta) \cdot r \\ x' &= \Delta x + \sin(\theta + \beta) \cdot r \\ y' &= \Delta y + \cos(\theta + \beta) \cdot r \end{aligned}$$

Else we can consider the vehicle is moving in a straight direction and cut down on unnecessary calculations

$$\begin{aligned}x' &= x + \sin(\theta) \cdot d \\y' &= y + \cos(\theta) \cdot d\end{aligned}$$

We then calculate the final change in angle

$$\theta' = \theta + \text{beta} \pmod{2\pi}$$

The end point of the motion primitive just calculated are then used to create a new node  $n$

$$n = (x', y', \theta')$$

#### 4.1.2 Open and Closed lists

As a reminder, the open list keeps track of all the nodes that have been expanded, the node with the lowest  $f(n)$  is removed from the list and used to further calculate the motion primitives, so in an ordinary list it would have to search through the list to find the node with the lowest  $f(n)$ , but by using a Binary Min-Heap data structure the best node is set to the top of the data structure, See figure 4.1. This results in a time complexity  $O(1)$  for finding the minimum  $f(n)$  node  $O(\log n)$  for deleting and inserting, comparing to that of the  $O(n)$  search, deletion and insertion of a standard list. I found an implementation of a min-heap online which I decided to use, I had to adapt the implementation, so it can store objects of my class "Node". The Heap must shift the Nodes when one is either inserted or deleted, to perform this operation the Heap must first compare if one node is greater than the other, to achieve this I had to redefine the relational operators into the class, and so a node is just considered than the other if its  $f(n)$  is larger than the others.

There are occasions where nodes have the same  $f(n)$  we want to perform a tie-break, this is where we choose the node with the highest  $g(n)$  from the two this is so that it encourages exploration that is closer towards reaching the goal [5].

The purpose of the closed list is to keep track of nodes that have been conclusively processed, this is then used to check if a cell has been already expanded by getting the discrete coordinates from the nodes within the closed

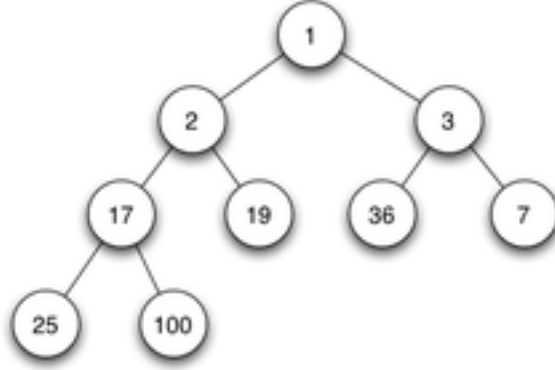


Figure 4.1: Example representation of a Binary Min-Heap

list. In my implementation I do not encode the closed list as a list of nodes which have been removed from the open list, rather I have a binary matrix of the exact dimensions of the grid used to keep track of track boundaries, this means the cell coordinates match 1 to 1 with the index of the matrix, I can then index into the matrix to find if a cell is expanded by checking for a value of 1. This difference from the standard implementation of hybrid a\* partly due to my initial miss-understanding of the function of the closed list and partly for a simpler method to visualize the expansion of the cells for debugging purposes. This miss implementation should not affect execution time, but the matrix will occupy more memory over a basic array when using a large map.

### 4.1.3 Heuristic and Cost

How close the output path is to the global optimum heavily relies on the heuristic used, and its properties, such as it being admissible heuristic meaning a heuristic that does not overestimate the actual cost to the goal.

My initial approach involved using Dubins path as my heuristic but turned out to be ineffective and problematic even when Dubins path was the actual solution, the path constructed from motion primitives could not reconstruct Dubins precisely enough due to the set distance of the motion primitives.

So I resorted to the simpler heuristic of Euclidian distance to the goal, this appears effective for when goals are placed in a direction that is ahead

from the initial direction of travel. When the goal is placed behind the direction of travel, the heuristic will encourage exploration for most of the initial stretch of the track, this also leads to the path taking unnecessary turns because it attempts to make a full 180 turn to reach the goal, See figure 4.2. The Euclidian distance can fail and increase execution time when the reverse of what the heuristic is suggesting is occurring, such that when the exploration gets closer to the goal the heuristic suggests that it is actually going further away from the goal, See table 5.3. This is where the implementing the additional cost for changing steering angles can have positive effect on the resulting path but at the cost of expanding more nodes, See figure 4.2 and 4.3.

Nodes are connected by motion primitives which are represented by the straight lines, which are a simplified representation of arcs. All the motion primitives that are currently visible are ones that have been added to the open list, in future images green lines will represent Dubins path. The small red circle is the starting point of the path, the goal is the larger blue circle, the red path is the final produced path, the black cells represent the track boundaries.

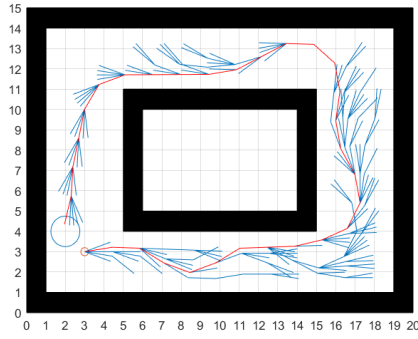


Figure 4.2: Example of path taking unnecessary turns at the start of the lap because of Euclidian distance heuristic

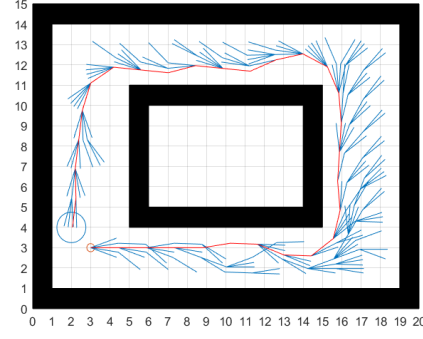


Figure 4.3: Algorithm from figure 4.2 adapted by adding additional cost for changing steering angles, larger change leads to larger  $g(n)$



#### 4.1.4 Defining the Goal

Initially I defined reaching the goal as a node with  $(x, y)$  coordinates and angle  $\theta$  and arriving at the goal simply meant checking if the chosen node is equal to the goal node, but it quickly became apparent that for a motion primitive to land into the exact coordinates and angle as the goal is extremely unlikely. This led to having the goal being an area of radius  $r$  where  $r \leq d/2$ ,  $d$  being the distance travelled along a motion primitive; this is so the motion primitive cannot surpass the goal. I wanted to maintain some range of angles that was acceptable, but due to the expansion of cells, See subsection 4.1.2, it would result in the goal being cut off from exploration if the goal was reached at the wrong angle. So, in the end I removed the restriction on angle upon entering the goal, this approach results in less control on the state of the vehicle upon arrival, my hypothesis is that when moving from goal to goal the path the path will suffer slightly

#### 4.1.5 Collision Detection

Checking for collisions is more than checking for the nodes landing within a constrained cell, since the dimensions of the vehicle are larger than that of the path itself, this means we cannot solely rely on the position of the node to check for collisions, we need to search for the closest obstacle. Iterating over each individual cell that is a boundary or obstacle to find the closest obstacle is too slow and ineffective. To achieve a faster result, I encoded the obstacle map into a k-d tree, which is a data structure typically used for organizing data points in a space with k dimensions. I could then use MATLABs statistics toolbox that contains the K-nearest neighbours function, to find the closest obstacle and the distance to it from a point, given the k-d tree containing the obstacle map. I can then check if the distance is within some defined range. In my implementation I decided to use a range that is larger than that of the length of the vehicle.

#### 4.1.6 Dubins path

Some times the shortest path to the goal is Dubins path, so checking to see if Dubins path is a valid solution before calculating motion primitives can result in large improvements in execution time. Part of the problem is that time is spent having to calculate Dubins path and check for collisions, and doing

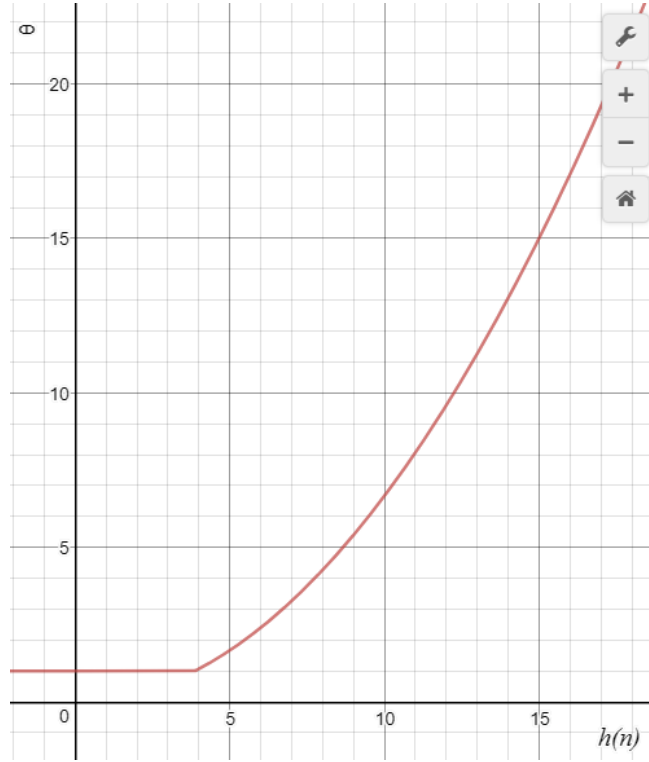


Figure 4.4: Dubins path threshold on the  $x$  - axis and threshold  $\theta_d$  on the  $y$  - axis set with  $w_d = 15$

so for each node can also be extremely costly in terms of execution time. So there is some trade off which needs to be considered; as we approach the goal is more likely that Dubins path will be a valid solution, this means we can have a threshold for when Dubins path is checked, which decreases as the goal is approached. How close we are to the solution is indicated by the heuristic. I assume the probability for Dubins path to collide with the track boundaries to be exponential the further it is away from the goal, so I will use the following equation to calculate the threshold  $\theta_d$ :

$$\theta_d = \max\left(\frac{h(n)}{w_d}, 1\right)$$

where  $w_d$  is a weight, a plot of this function is shown in figure 5.2.

A counter increases for each node that is extracted from the open list, and if it passes the threshold it is set back to 0. I downloaded the C++ code

for computing Dubins path and the mex function used to pipe the output to MATLAB. The function takes as arguments:

1. Coordinates and heading of the starting point
2. Coordinates and heading of the end point
3. Radius which is the radius of the circle of maximum angle
4. Step size of the path

I adapted the Mex so it would return the length of the path, originally used as the heuristic, but now it is an artefact after no longer being used. Because of the way Dubins path is implemented it reaches the goal at the heading specified,

#### 4.1.7 Minor notes

The track is stored as a black and white image, where black pixels represent the boundaries and white the traversable space, the image can be of any dimensions. I was not able to program the change of grid resolution in time, so in order to change the scale of the track would require manual change of constants and variables, which includes:

| Scaling variables                          |                        |               |
|--------------------------------------------|------------------------|---------------|
| Variable                                   | Location               | Variable name |
| Vehicle length                             | Node.m line 6          | vlen          |
| Distance travelled in one motion primitive | Node.m line 4          | d             |
| Collision radius                           | Node.m line 17         | cr            |
| Goal radius                                | hybrid_astar.m line 26 | goal_r        |

Table 4.1: Scaling variables

# Chapter 5

## Results and Evaluation

### 5.1 Specification

Windows 10 64-bit

Intel(R) Core(TM) i5-2500k @ 3.30GHz overclocked to 3.6GHz

| Variable values                                |           |
|------------------------------------------------|-----------|
| Dimension of cells                             | 1x1 meter |
| Distance of motion primitive                   | 1.45 cell |
| Vehicle length                                 | 1.5 cell  |
| Maximum steering angle                         | 35°       |
| Number of motion primitives per node           | 5         |
| Additional $g(n)$ for change in steering angle | 0.01      |
| Radius of goal                                 | 0.75 cell |
| Dubins path threshold weight                   | 25        |

Table 5.1: Testing the performance when reaching goal around a turn at different sizes

### 5.2 Finding optimal distance between waypoints

For this section I will attempt to find the breaking points when choosing distance between waypoints. By recording the execution times, I can inter-

polate between the points to find a line of best fit, which I can then use to estimate the optimal distance between waypoints depending on the choice of what is the minimal distance that a vehicle can travel for before the new generated path. This will vary as execution time will change depending on different configurations of a track, on a straight path there are not as many variables to consider, and since the path is straight I can easily control the final length of the path.

### 5.2.1 Straight track

| Straight start to end with no change in direction |            |            |            |                  |                |                      |
|---------------------------------------------------|------------|------------|------------|------------------|----------------|----------------------|
| Distance along track (cells)                      | Time 1 (s) | Time 2 (s) | Time 3 (s) | Average Time (s) | Nodes expanded | Steps in Dubins path |
| 5                                                 | 0.007      | 0.008      | 0.007      | 0.007            | 2              | 3                    |
| 10                                                | 0.010      | 0.011      | 0.009      | 0.010            | 3              | 4                    |
| 25                                                | 0.041      | 0.043      | 0.041      | 0.042            | 10             | 11                   |
| 50                                                | 0.151      | 0.165      | 0.161      | 0.159            | 24             | 25                   |
| 100                                               | 0.500      | 0.504      | 0.526      | 0.510            | 52             | 53                   |

Table 5.2: Testing time executions for different lengths of straight drives

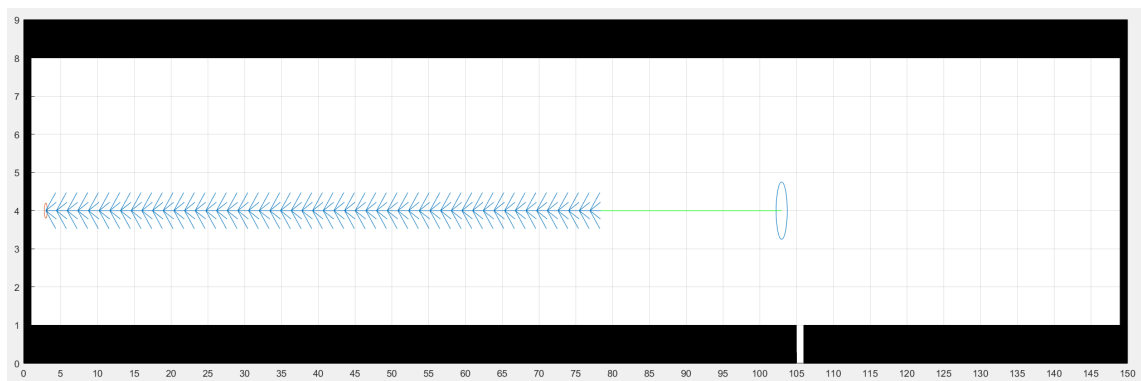


Figure 5.1: Display of the algorithm going straight through 100 cells, dubins path is the green line

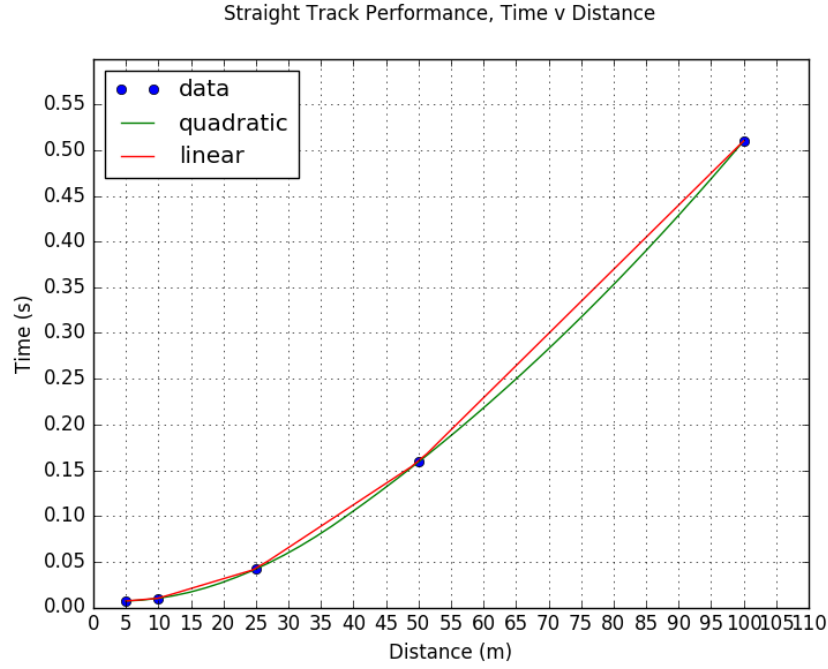


Figure 5.2: Interpolation of averaged execution times against the path length

## 5.3 Testing performance

Here I will test the algorithm on different tracks, also testing the effectiveness of the Dubins path extension, If there are any scenarios in where it under performs.

### 5.3.1 Circular track without Dubins path ext

This track is used to test the performance of the algorithm when trying perform turns.

For tracks that have large turns such that of figure 5.6 hybrid a\* will explore a lot of the search space before reaching the goal, but will still produce decent results with paths with small turns such as 5.7.

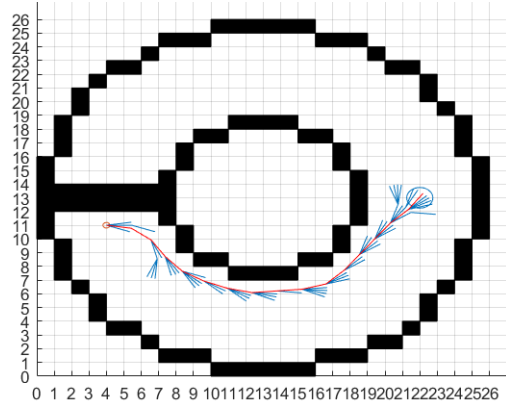


Figure 5.3: Result and exploration path finding for half of circular track (no Dubins path)

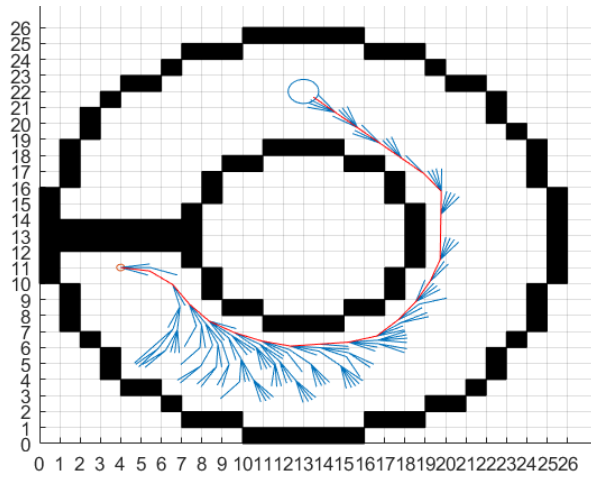


Figure 5.4: Result and exploration path finding for three quarters of circular track (no Dubins path)

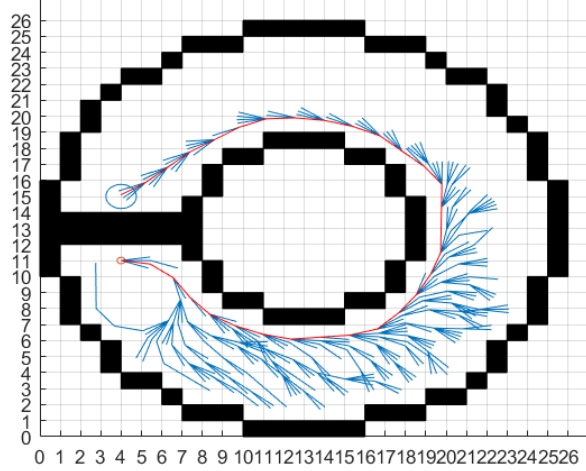


Figure 5.5: Result and exploration path finding for full circular track (no Dubins path)

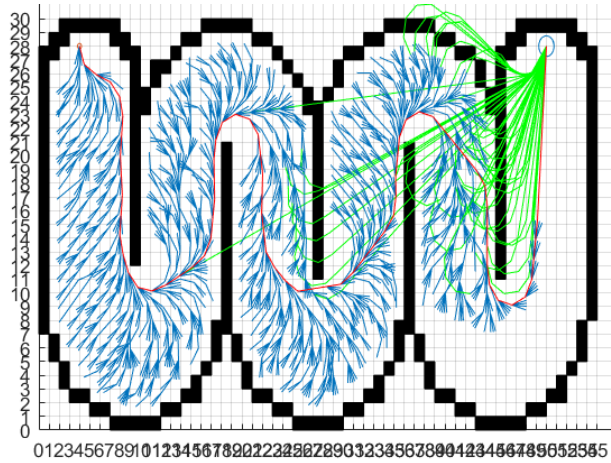


Figure 5.6: The algorithm is able to find a path to the goal but this is after searching for large portions of the search space, due to the placement of the goal and the large turns. The time required to generate this path was 4.38s. The green paths are Dubins paths which are being checked.



| Circular track - no Dubins path |            |            |            |                  |                          |                                |        |
|---------------------------------|------------|------------|------------|------------------|--------------------------|--------------------------------|--------|
| Track section                   | Time 1 (s) | Time 2 (s) | Time 3 (s) | Average Time (s) | Nodes added to open list | Distance Travelled (Cell size) | Figure |
| 1/2 track                       | 0.068      | 0.051      | 0.046      | 0.055            | 29                       | 24.65                          | 5.3    |
| 3/4 track                       | 0.154      | 0.162      | 0.175      | 0.164            | 112                      | 34.80                          | 5.4    |
| full track                      | 0.325      | 0.353      | 0.334      | 0.337            | 242                      | 44.95                          | 5.5    |

Table 5.3: Testing the performance when reaching goal around a turn at different sizes

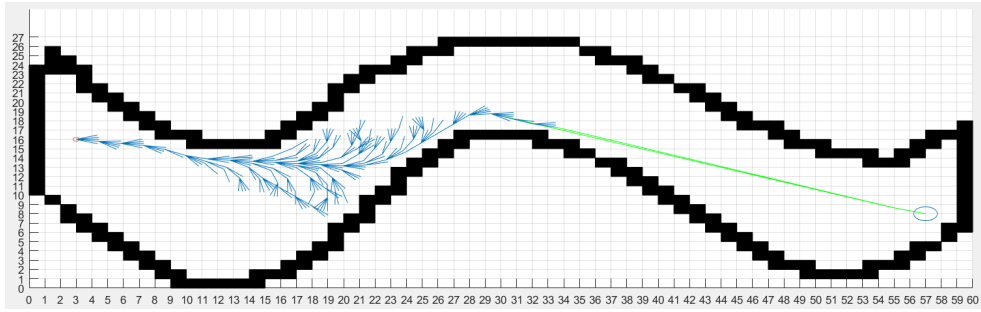


Figure 5.7: The algorithm is able to find a path to the goal with some minor unwanted exploration when reaching the turn. See figure 4.1.3 for explanation on interpreting the figure

## 5.4 Fail case

If the goal is ever placed in an unreachable position either because of tight turns or it being placed too close or inside an obstacle, then the algorithm will not be able to recognise it has failed to reach the goal and will continue exploring the search space until it has explored everything it can explore. This can be a big issue if the goal is ever miss-placed or there is some edge case where having the goal placed away from, and as this will be operating in an incomplete map of the track it will start searching outside of the track boundaries and get caught in a standstill, See figure 5.8.

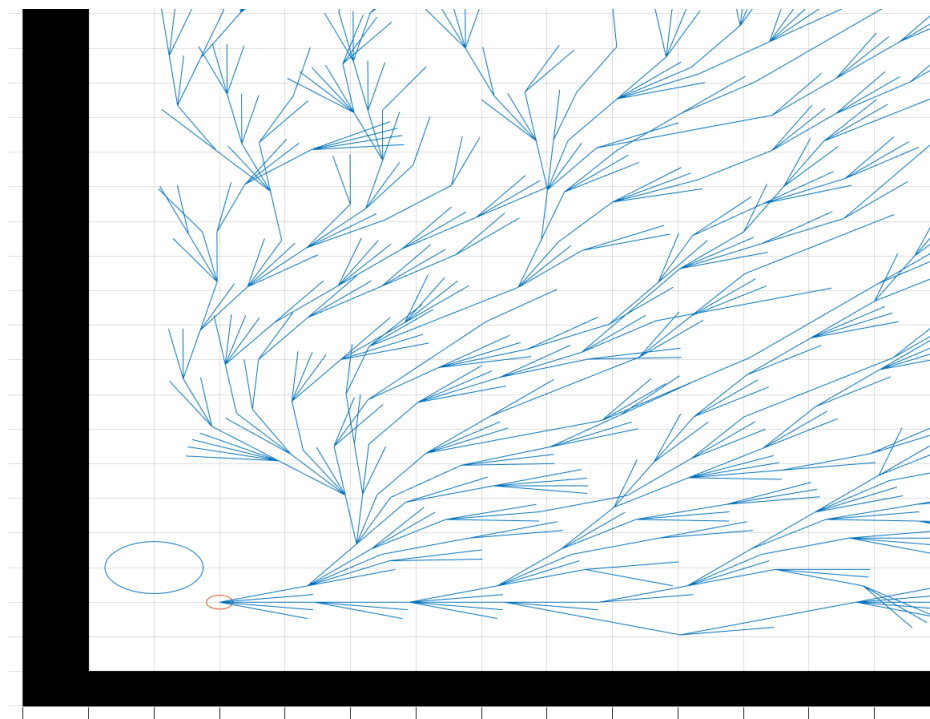


Figure 5.8: display of continuous exploration of the search space even if the goal cannot be reached

# Chapter 6

## Future Work

There are many different improvements I would of liked to implement over the course of the project, and due to lack of time, unseen difficulties and problems, I was not able to achieve everything stated in the introduction. Work on the project will not directly conclude after the project submission, I will continue to work on improving my current implementation.

### 6.0.1 First focus

My first focus will be in connecting the simulated environment to MATLAB and Implement a basic controller, this will allow me to better test the quality of the produced paths, get a feel for whether what I've implemented is fast enough and allow me to implement a method for placing waypoints with the expanding model.

### 6.0.2 Second focus

The second main focus follows my hypothesis that the best improvement for the performance of my implementation of hybrid a\* will be attempting to improve how the heuristic is calculated, aiming to achieve what is illustrated in figure 3.1.

### 6.0.3 Other potential improvements

All the work done so far in aim of developing an implementation for making the first lap of the track, there is still alot of work to be done for an optimal

path planner optimized for racing, one approach would involve reusing the algorithm used to generate the path, smoothen the path generated and create a velocity profile, which would save alot of time and I would assume less research than reimplementing another algorithm.

# Chapter 7

## Conclusions

The final produced result only partially solves the goals aimed for in the introduction, my implementation of hybrid-a\* algorithm has provided a partial solution to the problem of navigating a vehicle through the first lap. My goal of running a simulator was not accomplished, whilst the solution is functional, it is not yet practical nor can be full incorporated with the simulated environment. Currently the goals are manually set as well as many variables that still require fine tuning and adjustments, the solution also requires the implementation of waypoints being programmatically placed inside a simulation of a growing map.

# Chapter 8

## Reflection

There are a lot of things that I have come to learn over the course of working on this project. I came into the project with large ambitions and little experience to no experience with the tools I was going to use. My initial largest challenge was coming to read and comprehend the content within research papers that was related to what I was trying to achieve, being quite unfamiliar to the field of robotics I felt quite overwhelmed by the quantity of complex material presented and mathematical equations. Although I was familiar with most of the notation, keeping track of what every variable represents so that I can truly grasp the meaning of the equation, so I can then understand how relevant it is to my project, and then translating it to something I can implement was a big challenge. Researching consumed a lot of time, so spending time earlier in the phase of developing the project is important, spending more time trying to explain what I've managed to grasp from what I've read to my supervisor. Often, I had to revisit the content of the paper for things that I would have missed or miss-understood.

I expected a big challenge attempting this project which I received and was not able to handle, as I underestimated the difficulty of what I was attempting, laying out what I am capable of and being more open to criticism from my supervisor would help. Before starting the project, I had little knowledge or experience using MATLAB and its IDE, I've become confident at coding in it. Thinking about writing the outline to the report at an early stage during the development phase would have helped me lay out my thoughts and grasp the size of the project and report that I was about to face.

# Bibliography

- [1] E. Szádeczky-Kardoss and B. Kiss, “Extension of the rapidly exploring random tree algorithm with key configurations for nonholonomic motion planning,” in *Mechatronics, 2006 IEEE International Conference on*, pp. 363–368, IEEE, 2006.
- [2] J. Petereit, T. Emter, C. W. Frey, T. Kopfstedt, and A. Beutel, “Application of hybrid a\* to an autonomous mobile robot for path planning in unstructured outdoor environments,” in *Robotics; Proceedings of ROBOTIK 2012; 7th German Conference on*, pp. 1–6, VDE, 2012.
- [3] M. Montemerlo, J. Becker, S. Bhat, H. Dahlkamp, D. Dolgov, S. Ettinger, D. Haehnel, T. Hilden, G. Hoffmann, B. Huhnke, *et al.*, “Junior: The stanford entry in the urban challenge,” *Journal of field Robotics*, vol. 25, no. 9, pp. 569–597, 2008.
- [4] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, “Practical search techniques in path planning for autonomous driving,” *Ann Arbor*, vol. 1001, no. 48105, pp. 18–80, 2008.
- [5] A. Patel, “A\*’s use of the heuristic,” *Website: <http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html>*, 2007.